MotionBench: Agent-Independent Robot Evaluation through Goal-Referenced Outcome Scoring

An Open Framework and Preliminary Evaluation in Simulation

Abhay Yadav

Independent researcher, San Francisco Bay Area, California, USA [contact email]

https://motionbench.org https://github.com/motionbenchmark/mb-core

Draft v0.5, October 2026

Abstract

Comparing human-operated and autonomous robots is difficult when their control methods differ. In student robotics competitions, for example, match scores do not show whether a programmed routine performs better than a human driver under the same conditions. This paper introduces MotionBench, an open framework for evaluating robot task performance from externally observed trajectories and task-state measurements, without access to the agent's controller. MotionBench defines task-specific goals, reports the share of the goal an attempt achieved beside the level reached by chance, and specifies a protocol for comparing agents over repeated trials and previously unseen layouts. The framework is implemented as a deterministic scorer, trajectory-capture tools and a simulator that runs VEX AIM Python routines unchanged. In a preliminary simulated study of an object-sorting task, a routine that encodes the public layout scores 4.00 of 6 points there but 2.80 on unseen layouts, while a camera-based routine scores 2.45 and 2.35. These results illustrate the method; they do not establish real-world measurement accuracy or any comparison with real people, which remain to be tested. The implementation and protocol are released so that the method can be examined and validated on physical robots.

1 Introduction

In VEX robotics competitions, my team spends a lot of time improving autonomous routines. But I kept coming back to a basic question: was our code actually better than driving the robot ourselves? We could compare match scores, but those depended on the field, the opposing alliance and everything else that happened in the match. They did not tell us how much of a task each approach could complete under the same conditions.

I ran into the same problem working with friends on RoboBench, a project in which large language models write VEX robot code, the code runs on a robot, and the results are fed back to improve the next attempt. We needed a consistent way to say whether one attempt was better than another, without depending on how the code had been written.

MotionBench grew out of that problem. Instead of evaluating the controller, it measures the observable outcome of a task: where the robot went and what state it left the task in. The same scoring procedure can then be applied to a human driver, a programmed routine, AI-generated code or a commercial robot, provided the task and operating conditions are comparable.

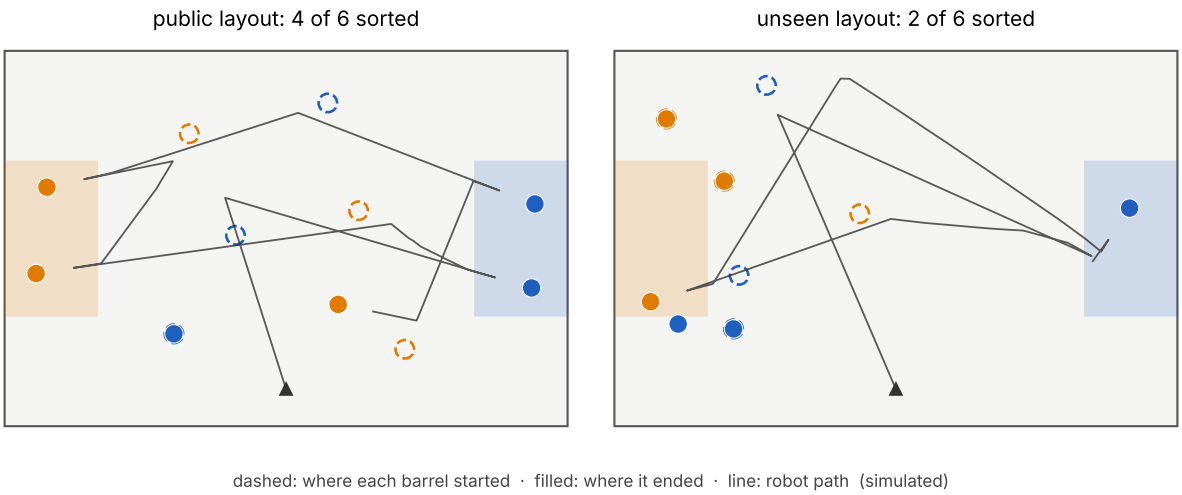


Figure 1: **The same routine on two layouts (simulated).** A VEX AIM routine that plans from the barrel positions of the public layout sorts 4 of 6 barrels there (left) and 2 of 6 on an unseen layout (right), where it drives to where barrels used to be. MotionBench scores both runs the same way, from the robot's path and where the barrels ended up.

MotionBench investigates whether robot performance can be evaluated independently of the controller, from externally observed task outcomes. Its design objectives are that the evaluation be fair (every agent scored by the same rules on the same task), repeatable (the same recording always gives the same score, and recordings of the same behaviour give similar scores) and cheap enough to run with a phone camera. This paper develops the scoring framework and protocol, demonstrates them in simulation, and identifies the real-world validation needed to establish measurement accuracy and repeatability; it does not claim to have established those objectives.

What the paper does claim holds by construction: every agent is scored from an externally observed trace and nothing else, by one explicit, deterministic and open scorer. The scoring parameters on top of that (goal definitions, tolerances, budgets, the equivalence margin, the weights of the secondary index) are a first proposal. They are versioned, reported with every result and changed only through public RFCs, so that results stay comparable within a specification version while the specification itself is revised as evidence arrives (Table 4). A MotionBench score is therefore a measurement under a stated specification, not yet a validated estimate of real-world performance. The contributions are:

1. a **goal-referenced scoring model** that reports the share of a task's goal achieved, beside chance, with comparisons to people only where people use the same means, and an **evaluation protocol** with public and unseen layouts, paired comparisons and explicit outcomes that separate "no difference found" from "equivalent" (Section 3);
2. an **open implementation and released benchmark** whose only interface to an agent is a recorded trace of poses and task state: a deterministic scorer, capture tools, a simulator, six task specifications and released data (Section 4);
3. a **preliminary simulated study** that runs the protocol end to end on a classroom sorting task (Sections 5 and 6).

The simulated study is illustrative; within it, I observe that:

(1) **A public-layout advantage did not carry over.** The routine that encodes the public

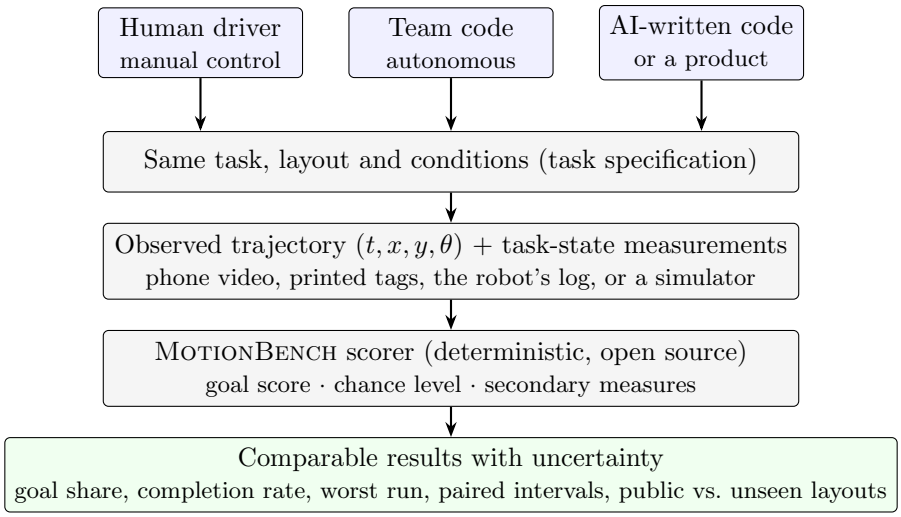


Figure 2: The MotionBench framework. Agents are never integrated with the benchmark: the scorer sees only what was observed during the attempt, and every agent is scored by the same rules.

layout scores 4.00 of 6 there, ahead of the typical simulated driver, but 2.80 on unseen layouts, with no difference found from that driver (Figure 1).

(2) **Its loss is barrels it never finds.** On unseen layouts it never picks up 60 of 120 barrels, against 20 on the public layout.

(3) **Most missing points, for every agent, are barrels never reached in time,** which suggests the 45 s budget, not sorting skill, limits scores on this task.

(4) **The time budget can change which code looks best.** Re-scoring the same runs at shorter budgets, the best routine on unseen layouts is the vision routine at 25–30 s and the planner at 35–45 s.

(5) **Capture is accurate to about 1 cm with printed tags and 3 cm without** on synthetic video; real footage is untested.

Figure 2 shows the framework, and Table 1 separates what is built, what has been demonstrated (in simulation or on synthetic data) and what is not yet shown.

2 Related work

Benchmarks for robot policies. Meta-World [17], CALVIN [11] and LIBERO [10] evaluate learned manipulation policies in simulation; Habitat [15] does so for embodied navigation; SimplerEnv [9] studies how well simulated evaluation predicts real-robot results. RoboDojo [5] combines simulated and real tasks on standardized bimanual hardware, scores real trials by blinded human evaluators with partial credit, and verifies submissions on hidden layouts. RoboArena [3] distributes real-world evaluation across institutions through pairwise comparisons. Evaluating task completion, standardizing tasks and using held-out layouts are not new; MotionBench adopts explicit protocols, hidden layouts, repeatability reporting and a separation between verified and unverified results from this literature. What it changes is the unit being evaluated: the whole agent (robot, software and any human operator), observed from outside. Table 2 compares named

Built and released	**Demonstrated (simulated or synthetic)**	**Not yet shown**
Scorer, trace format and task specifications (Apache-2.0 / CC BY 4.0)	The scorer is deterministic; published boards rebuild byte-identically from their traces	That task specifications capture what practitioners mean by "done"
Phone-video capture, with and without printed tags	Synthetic video: about 1 cm median error with tags, about 3 cm without	Accuracy on real footage (reflections, blur, occlusion, camera motion)
VEX AIM simulator running unmodified routines; parameterized simulated drivers	Simulation: a public-layout advantage can vanish on unseen layouts	That the simulator matches a real AIM robot; that simulated drivers resemble real students
Protocol: tiers, run counts, unseen layouts, paired comparisons	— (the verified tier is proposed and has not been exercised)	Repeatability of real sessions; a controlled comparison of real drivers and their code
Physical measurements in this paper: one hand-held video of a robot vacuum (no ground truth) and 120 motion-capture walks from an open dataset [16]. Everything else is simulated or synthetic.		

Table 1: Status of the work.

Benchmark	Setting	What is evaluated	Runs the agent's software	Scores human operators	Score
Meta-World [17]	simulation	learned policies	yes	no	success rate
CALVIN [11]	simulation	learned policies	yes	no	tasks completed in a row
LIBERO [10]	simulation	learned policies	yes	no	success rate
Habitat [15]	simulation	embodied agents	yes	no	success, path-length weighted
SimplerEnv [9]	simulation of real setups	learned policies	yes	no	success rate
RoboDojo [5]	simulation and real	generalist policies	yes	no	partial credit, blinded raters
RoboArena [3]	real, distributed	generalist policies	yes	no	pairwise preferences
IEC 62929 [1]	laboratory	cleaning-robot products	no	no	standardized test measures
MotionBench (this work)	simulation and real	whole agents	no	yes, where comparable	share of the task's goal

Table 2: MotionBench and existing evaluations. "Runs the agent's software": the benchmark executes or serves the agent's policy, rather than observing a run from outside.

benchmarks on a few properties, without claiming that they could not be extended in the same direction.

Human reference points. Human-normalized scores were popularized by the Arcade Learning Environment [4, 12]. Section 3 explains why MotionBench moved from normalizing to people to a goal-referenced score.

Reporting under uncertainty. Agarwal et al. [2] show that point estimates over few runs mislead and recommend interval estimates. MotionBench uses bootstrap intervals [6], paired designs and an equivalence margin in the spirit of two one-sided tests [8].

Product test methods. IEC 62929 [1] specifies laboratory methods for household cleaning robots. MotionBench is complementary: it aims to measure robots where they are used, from ordinary video.

Task	Platform	Goal	Arena	Budget	People comparable	From video today	Used here
MB-VEX-NAV@1.0	VEX V5	6 waypoints, 5 cm / 5°	3.66 m square	20 s	yes	tags[a]	index (App. B)
MB-VEX-PICK@1.0	VEX V5	deliver 6 objects	3.66 m square	60 s	yes	no[b]	index (App. B)
MB-AIM-NAV@0.1	VEX AIM	5 waypoints, 3 cm / 10°	1.83 × 1.22 m	20 s	yes	tags[a]	—
MB-AIM-SORT@0.1	VEX AIM	sort 6 barrels by colour	1.83 × 1.22 m	45 s	yes	no[b]	main study
MB-COVER@0.1	floor robots	cover free floor, 30 cm tool	4 × 3 m	180 s	no	yes	index (App. B)
MB-FIELD-HALLWAY@0.1	robot vacuum	cover visible floor	1.3 × 3.0 m	137 s	no	yes	demonstration

Table 3: Current task specifications. Only MB-AIM-SORT has a controlled (simulated) experiment; the others are implemented and used only for the demonstrations indicated. "People comparable": a person can do the same job with the same means, for example by driving the same robot. "From video today": whether the released capture tools can produce the trace from phone video. [a]Heading tolerances need printed tags; tag-free capture takes heading from the direction of travel. [b]Delivery needs final object positions, which the capture tools do not yet measure; these tasks can be scored today from simulators or from object positions entered by hand.

3 The MotionBench framework

3.1 Tasks and observations

What is measured. MotionBench scores task outcomes from externally observable trajectories and task-state measurements, without access to the agent's controller. A trace holds timestamped poses (t, x, y, θ) of the robot, optional events (for example "target knocked down") and, for tasks that move objects, the final positions of those objects. The scorer combines these with the task specification (arena geometry, robot footprint, goal, budget). For waypoint and coverage tasks the goal depends on the robot's path alone; for delivery tasks it depends on where the objects ended up, which the robot's path cannot determine and must be measured separately.

Tasks. A task specifies an arena (size, obstacles, zones), the robot footprint, a time budget, public layouts, rules for generating unseen layouts, and a goal of one of three kinds: *waypoints* (reach each in order within position and heading tolerances; a waypoint counts when a recorded sample lies within both tolerances, with no interpolation between samples, so a near-miss scores nothing), *delivery* (each object rests in its target zone at the end) or *coverage* (share of free floor swept by a tool of given width). Table 3 lists the current task specifications and which of them are used in this paper; they use competition hardware but never copy a season's game. A task carries its own version (after the @); the trace format and scoring rules are versioned by the specification (0.2), and the software by the scorer version (0.1.0). Every report records all three.

3.2 Scoring

Primary outcome: goal score. For a run with success S in task units and maximum $S_{\max}$, the goal score is $g = S/S_{\max} \in [0, 1]$. For each agent the board reports the mean over its runs with a 95% bootstrap interval, the worst run, and the *completion rate* (share of runs that achieved the whole goal). Time to goal is reported over runs that achieved it. The goal score is the headline and the only quantity used in this paper's comparisons.

Chance. Each board reports a chance level beside the agents' scores, without subtracting it. Chance is the median, over agents of class *baseline* in the board's data, of their mean success, expressed as a goal share like every other score; without a baseline agent it is the task's declared floor (0 for every current task). The baseline agent of the synthetic boards is a motion-level random walk: from its current position it draws a target from a normal distribution (standard deviation

Fixed by design	Provisional, versioned, changed by RFC
Agents are scored only from an externally observed trace (poses, events, final task state)	Goal definitions, tolerances and budgets of each task
One open, deterministic scorer for every kind of agent	The equivalence margin (±0.5 points for sorting)
The headline is the share of the task's goal, with chance shown beside it	The chance policy (a motion-level random walk)
Results carry tier, capture method and specification version, and are never mixed unlabelled	The secondary measures and the weights of the experimental index
Public and unseen layouts; every attempt reported	Run counts, number of unseen layouts, people-comparison guard (10%)

Table 4: What the framework fixes and what it treats as a first proposal. Results under an older specification remain valid as results under that specification.

0.6 m per axis, clipped to the arena), drives to it in a straight line at a fixed speed (0.2–0.6 m/s depending on the robot), pauses 0.2–1.0 s, and repeats until the budget ends. It sees only its own position and never manipulates objects, so on delivery tasks chance is 0 unless an object starts in its zone. On coverage a random walk already covers about half a room, which readers should see rather than have absorbed into a transformation. A random policy at the task level (for example one that picks up objects and drops them in random zones) would give a higher, and for some purposes more useful, reference; it is not implemented. No random agent was run in the sorting study of Section 6, whose declared floor is 0.

Comparison with people (optional). The first specification normalized every agent to the median person on the task, following [12]. That turned differences in equipment into apparent differences in skill when means differed (a robot vacuum against a person with a stick vacuum), left tasks without a human baseline unscored, and saturated where most people finish. Specification 0.2 reports vs_people $= (S - S_{\text{chance}})/(S_{\text{people}} - S_{\text{chance}})$ only on tasks declared people-comparable, only when people's median beats chance by at least 10% of $S_{\max}$ (otherwise the ratio is unstable), and never as the headline.

Secondary measures. Boards also report path efficiency, consistency and safety, defined precisely in Appendix B. Path efficiency compares the path driven with a simple reference length computed by the scorer (straight lines through the waypoints reached; a greedy nearest-first single-carry tour for deliveries; covered area divided by tool width for coverage). The reference ignores obstacles and robot kinematics, so it is a yardstick, not a shortest path. An optional index combines these measures with the goal score for ranking within one task. Combining them is a value judgement with known problems: consistency rewards an agent that always scores 1 of 6 over a better agent with varied results, and one safety violation means different things on different tasks. The index is therefore secondary, never averaged across tasks, not used in this paper's conclusions, and can be ignored; its definition and weight sensitivity are in Appendix B.

Ranks. For display, agents share a rank while their intervals overlap the leader's. This is a presentation convention, not a test of equivalence.

3.3 Evaluation protocol

Tiers. Results are *simulated* (S), *field* (F: a real robot captured and submitted by its owner, reviewed before publication) or *verified* (V). They are never mixed without labels. The verified tier is a proposed procedure that has not yet been carried out: it requires a layout sent at the start, one uncut fixed-camera video with a code word, every attempt reported, and a second reviewer re-scoring at least one run from raw video; reviewers declare conflicts of interest (details in the protocol document in the repository).

Runs and layouts. An agent needs at least five runs on a task to be ranked (ten recommended); runs are consecutive attempts and every attempt is reported. Public layouts are published; unseen layouts are generated from each task's layout rules with an unpublished seed.

Paired comparisons. In simulation, run i of every agent uses the same seed, so the same layout and the same draw of robot imperfections, and a comparison is the mean of paired differences. Intervals are 95% percentile bootstrap intervals from 4,000 resamples. On one layout the resampling unit is the paired run; across unseen layouts the bootstrap is hierarchical (layouts, then runs within each chosen layout). Paired simulated runs share the simulator and its assumptions, so they are not independent experimental replicates in the sense of separate physical sessions.

Outcomes. *Better* or *worse* when the interval excludes zero; *equivalent* when the interval lies strictly inside a margin of $\pm\delta$; *no difference found* otherwise. Requiring a 95% interval inside the margin is a conservative version of the two one-sided tests procedure [8], which uses a 90% interval. For the sorting task $\delta = 0.5$ points, half of the smallest scoring unit (one barrel), fixed before the study; it is a convention, not derived from what matters to users, and other tasks will need their own. "No difference found" does not mean equivalent.

4 Implementation and released benchmark

Traces and the scorer. A run is a directory with a manifest (task, layout, agent and its class, capture method, tier, file hashes), timestamped poses, optional events and final object positions. This is the only interface between an agent and the benchmark. The scorer is implemented once in Rust and called from Python and the command line; it validates traces (hashes, monotonic time, sampling gaps), computes run reports and builds boards. A separate recount, written in JavaScript in the toolkit's report page, recomputes waypoints, deliveries and coverage from raw poses and flags disagreements with the scorer. Both were written with the same AI assistant, so they are not fully independent checks. Seventeen hand-worked cases (`docs/handworked-cases.md`) state expected results derived by hand from the task rules, with the arithmetic written out, and cover boundary conditions: positions just inside and outside a tolerance, a wrong heading, waypoints out of order, a waypoint reached exactly at the time limit and one after it, a gap in the recorded samples, an object exactly on a zone edge and one just outside, the delivery reference length, consistency, chance, the people-comparison guard, a single coverage pass and wall-contact episodes. All seventeen pass. When first written, two of the hand derivations were wrong and the scorer was right; both were corrected. Seven properties that hold by construction are also checked on 40 random traces each (`python/tests/test_properties.py`): the same trace always gives the same report; goal share and path efficiency stay in $[0, 1]$; the agent's class, identifier and name do not change a run's score; shifting every timestamp by a constant does not change it; appending samples

Set	Task	Layouts	Agents	Runs	Source
synthetic-v0.1	MB-VEX-NAV@1.0	2	20	200	simulated
	MB-VEX-PICK@1.0	1	20	100	simulated
	MB-COVER@0.1	1	10	30	simulated
	MB-AIM-NAV@0.1†	2	16	160	simulated
	MB-AIM-SORT@0.1†	1	16	80	simulated
field-v0.1	MB-FIELD-HALLWAY@0.1	1	1	1	real (phone video)
AIM sorting study	AIM-SORT game	1 + 10	11	440	simulated (regenerated by script)
match-sim-v0.1	2-versus-2 matches	—	4 per match	5 matches	simulated

Table 5: Released data. †Marked invalid on the project site (simulated agents faster than the AIM robot can drive) and not used in this paper. The study's 440 runs (4 routines and 7 simulated drivers, 20 public and 20 unseen runs each) are regenerated by `paper/reproduce.sh`.

never lowers a waypoint score; extending a path never lowers coverage; and moving one object into its zone raises a delivery score by exactly one. Returning the same report for the same trace says nothing about whether the trace or the task specification is right; capture accuracy and task design are examined separately.

Capture. With printed AprilTags [13] in the arena corners and on the robot, a per-frame homography maps the image to the floor. Without tags, frames of a fixed or hand-held camera are aligned to the first frame; the per-pixel median gives an empty-floor image and the robot is the region that differs from it. The floor plane comes from four image points outlining a floor rectangle [7]; focal length and scale come from a measured width and length, or from the robot's known size. The robot's position is taken half a robot behind the lowest point of its silhouette, and its heading from its direction of travel, so tag-free capture cannot measure heading while the robot turns in place. Floor the camera cannot see is excluded from coverage and safety.

Simulation. A *game* is defined once (field, zones, wall-mounted fiducials, objects, layouts, layout rules, match length, scoring) and then simulated many times; a game may link to a task with the same layouts, in which case simulated runs are also scored by the standard scorer. The first platform is VEX AIM, a small holonomic classroom robot with a magnetic pick-up, a kicker and a camera. The simulator runs VEXcode AIM Python files unmodified: the robot API is replaced by stand-ins, every call advances simulated time, speeds are capped at the published API limits (200 mm/s, 150°/s), and each run draws its own wheel slip, drift, turn overshoot, kick scatter and camera noise. These are modelling assumptions; speed limits alone do not validate the simulator's dynamics, pick-up, kicking, perception or collisions against a real AIM robot. The *parameterized simulated-driver baseline* is a set of controller policies with explicit traits (reaction time with partial anticipation, share of full speed used, steering shake, placement precision, choice of next object, mistake and hesitation rates). Six named characters span a range of skill, and the *typical simulated driver* is the median member of a simulated class of twenty. The traits were chosen by hand to be plausible and are listed in Appendix C; they are not fitted to real students.

Released data. Table 5 lists what is released with this paper. All of it is labelled by tier and capture method; only the robot-vacuum run is real. Appendix E shows a task specification and a trace.

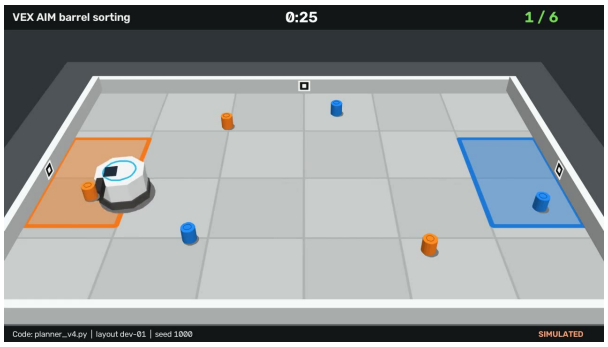


Figure 3: The simulated AIM sorting game on the public layout (labelled as simulation in all outputs).

5 Experimental setup

Task. One VEX AIM robot must sort three orange and three blue barrels into the zone of their colour (Figure 3) on a 1.83 m × 1.22 m table within 45 s (one point per barrel; six maximum). The public layout is `dev-01`; unseen layouts keep the same start and place the barrels at random away from the zones, the start and each other.

Agents. Four routines, written for the public layout, and seven simulated drivers, in four groups:

(1) **Open loop:** *dead reckoning*, a fixed plan with no sensors.

(2) **Sensing:** *vision*, where the camera finds the nearest barrel and wheel odometry returns it to the zone; and *vision + tags*, the same plus homing on the AprilTag behind each zone.

(3) **Layout-aware:** *planner*, which uses the barrel positions of the public layout, an efficient order and full speed, the camera only for the last few centimetres, and spread drop positions.

(4) **Simulated drivers:** six named characters and the typical simulated driver (Section 4; traits in Appendix C).

Runs and statistics. Every agent plays 20 paired runs on the public layout and 20 paired runs over ten unseen layouts (two per layout), with the paired bootstrap, the equivalence margin of ±0.5 points and the outcomes of Section 3.3.

Routine development and selection. The four routines were written with the AI assistant as examples of common approaches, one version of each (the suffixes v1–v4 number the routines, not revisions of one routine). They were debugged by running them on the public layout, which therefore served as a development layout, not an unbiased test set; for example, the planner's drop positions were spread out after simulated runs showed barrels being picked up again, and the tag routine's stopping point (how large the tag must appear before it releases) was raised. No alternative planners were tried. The driver controller was adjusted in the same period to remove steering oscillation caused by reaction delay (partial anticipation, a lower turning gain); the trait values of Appendix C were set by hand and not changed after any comparison was run. The unseen-layout generator was added after the routines were written, and no routine, driver trait or controller setting changed after unseen layouts were generated; the repository history records the

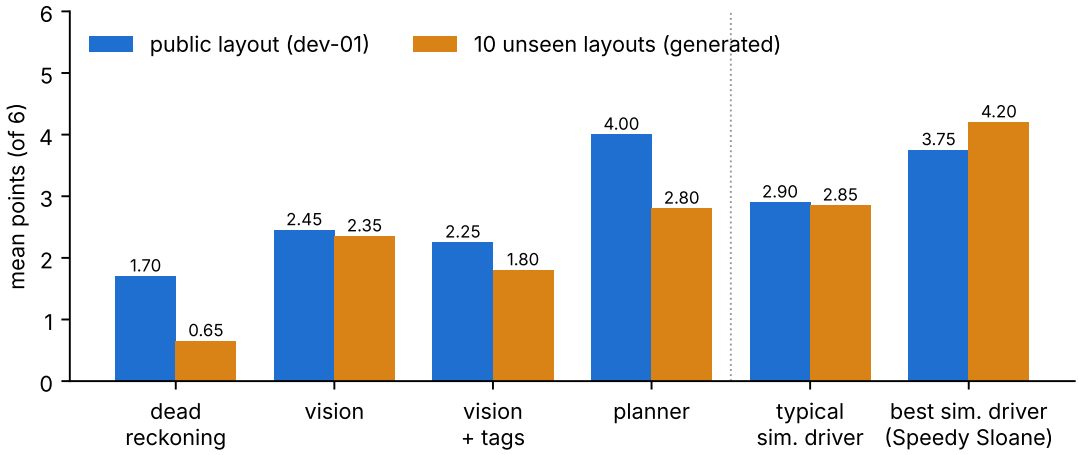


Figure 4: Mean points on the public layout and on ten unseen layouts (20 paired runs per agent, simulated). The drivers are a parameterized, uncalibrated baseline.

order. The planner is called the strongest routine because it has the highest public-layout mean, not because it was chosen from a larger set.

Capture accuracy. Separately, a robot's path is rendered as video with known ground truth, with printed tags (a fixed camera over a VEX field) and without (a hand-held-style view of a planked floor), over three render seeds, and recovered with the capture tools.

6 Results

All numbers in this section come from simulation or synthetic video and illustrate what the protocol reports.

6.1 Does a public-layout advantage generalize?

A public-layout advantage did not carry over. Table 6 and Figure 4 summarize the outcome. On the public layout the planner scores 4.00 points, ahead of the typical simulated driver (2.90) by a paired difference of +1.10 (+1.00 to +1.25). On unseen layouts it scores 2.80: a change of −1.20 (−1.60 to −0.85) and no difference found against the typical simulated driver (−0.05; −0.65 to +0.60). The vision routine scores 2.45 on the public layout and 2.35 on unseen layouts; its change (−0.10; −0.55 to +0.40) is small, but the interval is wider than the margin, so the outcome is no difference found, not equivalence. The dead-reckoning routine, which also encodes the public layout, changes by −1.05 (−1.50 to −0.60). Intervals are percentile intervals from 4,000 bootstrap resamples, reported to two decimal places; the discrete scoring scale and the small number of runs limit their numerical resolution.

Interpretation. The planner's public-layout advantage comes largely from knowing where the barrels are. That a routine which memorizes positions does worse when they move is expected; the point of the study is that the protocol catches it, and that a verdict based on the public layout alone would have recommended the planner. The within-routine change does not depend on the

Routine	Public layout		Unseen layouts		Change
	Mean	vs typical driver	Mean	vs typical driver	unseen − public
Dead reckoning	1.70	−1.20 [−1.50, −0.90] worse	0.65	−2.20 [−2.75, −1.55] worse	−1.05 [−1.50, −0.60]
Vision	2.45	−0.45 [−0.80, −0.10] worse	2.35	−0.50 [−1.15, +0.20] n.d.f.	−0.10 [−0.55, +0.40]
Vision + tags	2.25	−0.65 [−1.05, −0.30] worse	1.80	−1.05 [−1.55, −0.55] worse	−0.45 [−1.10, +0.15]
Planner	4.00	+1.10 [+1.00, +1.25] better	2.80	−0.05 [−0.65, +0.60] n.d.f.	−1.20 [−1.60, −0.85]
Typical simulated driver	2.90		2.85		

Table 6: Simulated AIM sorting: points (of 6) with 95% bootstrap intervals; n.d.f. = no difference found. The change column compares different layouts and is an unpaired bootstrap. The typical driver's paired runs average 2.90; the class median of per-driver means, used only to select that driver, is 2.83. Full results, including each driver character, are in `results/aim-sort-verdict-v0.1.json`.

Agent	Public layout					Unseen layouts				
	sorted	never picked	carried	misplaced	re-grabs	sorted	never picked	carried	misplaced	re-grabs
Dead reckoning	34	70	0	16	0	13	95	1	11	0
Vision	49	65	6	0	7	47	65	7	1	10
Vision + tags	45	60	15	0	21	36	70	13	1	19
Planner	80	20	20	0	0	56	60	4	0	2
Typical sim. driver	58	55	7	0	4	57	45	15	3	14

Table 7: **Where the points went** (simulated). What happened to each of the 120 barrels in an agent's 20 runs: sorted (scores), never picked up, still carried when time ran out, or misplaced (in the other zone or outside both). Re-grabs count pick-ups of a barrel the agent had already sorted. Counted from the runs' final states and events (`paper/analysis/failure_accounting.py`); no new runs.

simulated drivers. Several limits apply. First, the routines were debugged on the public layout and the planner is singled out for its public-layout score, so its public-layout margin is optimistic; the unseen layouts were not used for development or selection and give the less biased estimate. Second, ten unseen layouts with two runs each is a small sample of layouts, so the intervals may understate layout-to-layout variability. Layout by layout (Appendix D) the planner scores above the typical simulated driver on four layouts, the same on three and below on three; more layouts are needed before such a comparison could be relied on. Third, at the simulated top speed no agent sorts all six barrels in 45 s (best runs score 4 or 5). The goal score therefore measures partial progress under a tight budget rather than completion; this is a property of the task as specified, and whether the budget should be lengthened is an open question for the task's next version.

Where the points went. Table 7 accounts for every barrel. **For every agent, the largest loss is barrels never picked up,** from 45 of 120 for the typical simulated driver to 95 for dead reckoning on unseen layouts, which is consistent with the tight budget noted above. **The planner's drop on unseen layouts is almost entirely barrels it never finds** (20 never picked on the public layout, 60 on unseen ones). **Homing on tags costs re-grabs:** the vision + tags routine picks up barrels it had already sorted 21 times on the public layout and 19 on unseen ones, against 7 and 10 for vision alone. Dead reckoning ends with 16 barrels in the other colour's zone on the public layout. These categories describe the simulated runs; they are the kind of breakdown the scorer's events make possible, not findings about real robots.

Budget	25 s	30 s	35 s	40 s	45 s
τ with the 45 s ranking, public	0.58	0.69	0.73	0.81	1
τ with the 45 s ranking, unseen	0.80	0.84	0.94	0.98	1
Best routine, unseen	vision	vision	planner	planner	planner

Table 8: Re-scoring the same simulated runs at shorter budgets (11 agents, 20 runs each per setting; `paper/analysis/budget_sensitivity.py`). Kendall's τ compares agents' mean scores.

Capture	Scale from	Median error	90th percentile	Frames without position
Printed tags	tag geometry	1.2 cm	1.5 cm	0%
No tags	measured floor	3.2 cm	5.2 cm	8.2%
No tags	robot size only	3.2 cm	5.4 cm	8.2%

Table 9: Capture accuracy on rendered video (medians over three render seeds; `paper/analysis/capture_accuracy.py`).

6.2 Sensitivity to the budget and to the layouts

The scoring parameters are provisional, so it matters how much they move conclusions. Without new runs, every run of the study can be re-scored as if the match had ended earlier: a barrel counts if, at the last recorded instant before the cut-off, it is not held and lies in its own zone (at 45 s this rule agrees with the game's score in 425 of 440 runs; in the other 15, a barrel that lies in its zone at the last recorded instant is held by the robot in the game's final state, so the cut-off rule counts one barrel more). Longer budgets would need new runs and are not studied. Table 8 compares the ranking of the 11 agents at each cut-off with the ranking at 45 s. **Rankings move with the budget, and on unseen layouts the best routine changes:** the vision routine leads the routines at 25 and 30 s, the planner from 35 s. The leading agent overall does not change (the planner on the public layout, the fastest driver character on unseen ones). Between the public and the unseen layouts, the ranking of the 11 agents at 45 s agrees only at Kendall's $\tau = 0.66$. Both results argue for reporting budget sensitivity with a task and for unseen layouts in every verdict.

6.3 Capture accuracy on synthetic video

Table 9 reports per-frame position errors on rendered video with known ground truth, over three render seeds. With tags the robot is found in every frame with a median error of about 1 cm; without tags the median is about 3 cm whether scale comes from a measured floor or from the robot's size, and about 8% of frames have no position. Synthetic video lacks reflections, blur, occlusion and lighting changes, and whether these errors or the missing frames change goal scores (for example on a 5 cm waypoint tolerance) has not yet been measured. These figures are therefore a best case: errors on real footage are likely to be larger. They are not a validation.

6.4 Demonstrations on real data

A robot vacuum. A household robot vacuum was filmed for 137 s with a hand-held phone, without tags (Figure 5). The pipeline estimated 85% coverage of 3.24 m^2 of visible floor with a 30 cm cleaning width, over 18.6 m of driving. Because no independent reference measurement was available, this value is a demonstration of the scoring output, not a validated coverage estimate. The figure redraws the coverage with a separate Python implementation of the scorer's rule, which

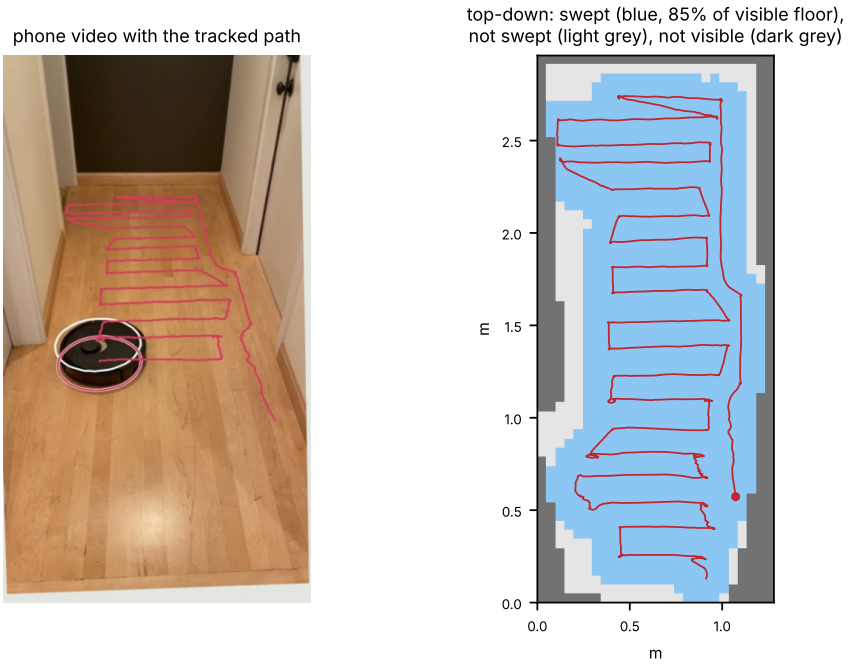


Figure 5: Tag-free capture of a household robot vacuum from one hand-held phone video. Left: a frame with the tracked path. Right: the same path in floor coordinates, with the floor swept by a 30 cm tool (blue), visible floor not swept (light grey) and floor the camera could not see (dark grey, excluded), as the coverage score uses them. There is no ground truth for this video (`paper/make_vacuum_fig.py`).

gives the same 85.0%.

People in an open dataset. I scored 120 motion-capture walks along one route (about 7 m) from THÖR-MAGNI [16]. All 120 reached the goal (median 6.8 s), so the goal score could not separate walkers; with a 60 s budget a random walk reached the goal in 40% of runs. The example shows that budgets need to follow typical completion times, or the goal score saturates.

7 Conclusion

MotionBench proposes evaluating any robot agent, whether a person, code or a product, by how much of a task's goal it achieves, from what can be observed from outside, with an open scorer and an explicit protocol. The framework and tools are built and released, and a preliminary simulated study shows why unseen layouts belong in any verdict and what a breakdown of lost points can reveal. The next step is a small real-world accuracy experiment that does not need a motion-capture lab: a robot driven between measured floor markers, 20–30 trials filmed with a fixed camera, comparing the position and waypoint completion estimated by MotionBench (with and without tags) against the measured marks, and checking whether capture error or missing frames change goal scores. Further work, in order: scoring the same simulated runs from ground truth and from rendered-video capture to test how capture error propagates to scores and rankings; more unseen layouts, at several difficulty levels, with layout-level reporting and a comparison of public-layout and unseen-layout rankings; running the routines of Section 5 on a real AIM robot to measure the simulator's prediction error; and a paired comparison of real student drivers with their own teams' code, which would require parental consent, a host institution's youth-protection rules and an ethics review. Comments are welcome at `https://motionbench.org/paper`.

8 Limitations

Simulation is not reality. The simulator is custom and uncalibrated: its robot model, noise and every driver trait are assumptions, and speed limits alone do not validate its dynamics, pick-up, perception or collisions. Simulated agents are less variable than real ones (some score the same in every run), so intervals are likely narrower than they will be on real robots.

Capture is validated only on synthetic video. Tag-free capture needs a visible floor rectangle and cannot measure heading during turns in place; the effect of capture error and missing frames on scores is untested. Delivery tasks need object positions, which the current capture tools do not measure.

Small samples of layouts. Ten unseen layouts with two runs each may understate layout-to-layout variability, and the routines were debugged on the public layout.

Task design. Goal scores depend on how goals and budgets are set. The sorting task's 45 s budget may be too tight, and coverage scores model cleaning rather than measure it. The synthetic VEX AIM boards published under specification 0.2 contain simulated agents faster than the AIM robot's top speed; they are marked invalid and are not used here. Tasks and protocol change only through public RFCs so that such issues are fixed openly.

AI-assisted implementation. The software was written with an AI assistant, and the scorer's separate recount was written with the same assistant. The hand-worked cases and the fresh-clone reproduction reduce, but do not remove, the risk of shared mistakes.

9 Broader impacts

MotionBench is meant to let students, teachers and researchers compare robot agents fairly, and to make claims about robots checkable. It also carries risks. *Ranking people:* scores of student drivers could be used to grade or select students, which the benchmark is not designed for; boards report agents, not people, and published human results use trajectories without names unless the person (or, for minors, a parent or guardian) agrees. *Video of people:* capture uses video of spaces where people may appear; MotionBench publishes paths, not footage, and videos showing people are not published without permission. *Over-trusting simulation:* a simulated verdict says which code to try on the robot first, not that code beats people; outputs label simulated results in every view. *Product claims:* comparisons of commercial robots measured in one home are not product ratings, and the field tier is reviewed before publication. *Studies with people:* no data from human participants were collected for this paper, so no ethics approval was sought. Any future pilot with people, and especially with minors, will need a host institution's ethics review (or an IRB determination); written consent from a parent or guardian and the participant's own assent; the option to opt out before any recording starts and to have data deleted later; random agent identifiers instead of names; and publication of trajectories only, without footage or metadata that could identify a home or a school.

Author contributions

A.Y. conceived the project, developed the research questions, designed the evaluation framework and the experiments, specified the intended behaviour of the software, ran the experiments and reviewed the resulting analyses.

Use of AI tools

AI-assisted tools, primarily Claude (Anthropic), used through claude.ai, supported software implementation (scorer, capture tools, simulators and analysis scripts), synthetic data generation, analysis scripting and manuscript drafting. AI-generated implementations and outputs were examined and tested by the author, who takes responsibility for the methods, reported results and conclusions. The checks made so far are the automated test suites in the repository (unit tests of the scorer and of the Python toolkit), the separate recount described in Section 4, and reproduction of every reported number with `paper/reproduce.sh`, and the seventeen hand-worked cases of Section 4, which were drafted with the assistant and whose arithmetic is written out so that it can be checked by hand. AI tools are not listed as authors.

Acknowledgements

The author thanks his friends and teammates on RoboBench and in VEX competition for the questions that started this work, his family for their support, and the THÖR-MAGNI authors for releasing their data.

Ethics, data and code availability

No new data from human participants were collected for this paper; the human trajectories come from a published dataset [16]. The robot-vacuum video was recorded by the author in a private home. Footage from a public event broadcast was used only to measure camera geometry for the match simulator (Appendix F) and is not redistributed. Code (Apache-2.0), task specifications and data (CC BY 4.0) are at `https://github.com/motionbenchmark/mb-core`; the version described here is the release tagged `paper-v0.5`, and `paper/reproduce.sh` regenerates every number and figure (Appendix A).

References

[1] IEC 62929: Cleaning robots for household use – dry cleaning: Methods of measuring performance. Technical report, International Electrotechnical Commission, 2014.

[2] Rishabh Agarwal, Max Schwarzer, Pablo Samuel Castro, Aaron Courville, and Marc G. Bellemare. Deep reinforcement learning at the edge of the statistical precipice. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021. arXiv:2108.13264.

[3] Pranav Atreya et al. RoboArena: Distributed real-world evaluation of generalist robot policies. In *Proceedings of the Conference on Robot Learning (CoRL)*, volume 305 of *PMLR*, 2025. arXiv:2506.18123.

[4] Marc G. Bellemare, Yavar Naddaf, Joel Veness, and Michael Bowling. The arcade learning environment: An evaluation platform for general agents. *Journal of Artificial Intelligence Research*, 47:253–279, 2013.

[5] Tianxing Chen, Yue Chen, Zixuan Li, et al. RoboDojo: A unified sim-and-real benchmark for comprehensive evaluation of generalist robot manipulation policies. *arXiv preprint arXiv:2607.04434*, 2026.

[6] Bradley Efron. Bootstrap methods: Another look at the jackknife. *The Annals of Statistics*, 7(1):1–26, 1979.

[7] Richard Hartley and Andrew Zisserman. *Multiple View Geometry in Computer Vision*. Cambridge University Press, 2 edition, 2004.

[8] Daniël Lakens. Equivalence tests: A practical primer for t tests, correlations, and meta-analyses. *Social Psychological and Personality Science*, 8(4):355–362, 2017.

[9] Xuanlin Li et al. Evaluating real-world robot manipulation policies in simulation. In *Proceedings of the Conference on Robot Learning (CoRL)*, volume 270 of *PMLR*, 2024. arXiv:2405.05941.

[10] Bo Liu, Yifeng Zhu, Chongkai Gao, Yihao Feng, Qiang Liu, Yuke Zhu, and Peter Stone. LIBERO: Benchmarking knowledge transfer for lifelong robot learning. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2023. arXiv:2306.03310.

[11] Oier Mees, Lukas Hermann, Erick Rosete-Beas, and Wolfram Burgard. CALVIN: A benchmark for language-conditioned policy learning for long-horizon robot manipulation tasks. *IEEE Robotics and Automation Letters*, 2022. arXiv:2112.03227.

[12] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, et al. Human-level control through deep reinforcement learning. *Nature*, 518:529–533, 2015.

[13] Edwin Olson. AprilTag: A robust and flexible visual fiducial system. In *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, 2011.

[14] Ergys Ristani, Francesco Solera, Roger Zou, Rita Cucchiara, and Carlo Tomasi. Performance measures and a data set for multi-target, multi-camera tracking. In *European Conference on Computer Vision (ECCV) Workshops*, 2016.

[15] Manolis Savva, Abhishek Kadian, Oleksandr Maksymets, Yili Zhao, Erik Wijmans, Bhavana Jain, Julian Straub, Jia Liu, Vladlen Koltun, Jitendra Malik, Devi Parikh, and Dhruv Batra. Habitat: A platform for embodied AI research. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2019. arXiv:1904.01201.

[16] Tim Schreiter et al. THÖR-MAGNI: A large-scale indoor motion capture recording of human movement and robot interaction, 2024. arXiv:2403.09285.

[17] Tianhe Yu, Deirdre Quillen, Zhanpeng He, Ryan Julian, Karol Hausman, Chelsea Finn, and Sergey Levine. Meta-World: A benchmark and evaluation for multi-task and meta reinforcement learning. In *Proceedings of the Conference on Robot Learning (CoRL)*, 2019. arXiv:1910.10897.

Appendix overview

Appendix A: reproducing the results and implementation details. Appendix B: secondary measures and the experimental index. Appendix C: simulated driver parameters. Appendix D: unseen-layout results, layout by layout. Appendix E: an example task specification and trace. Appendix F: multi-robot matches.

A Reproducing the results

Release: `paper-v0.5` of `https://github.com/motionbenchmark/mb-core`. Environment used: Python 3.13, NumPy 2.5, SciPy 1.18, OpenCV 5.0, FFmpeg 6.1, Rust 1.97, motionbench 0.1.0 (specification 0.2); the exact versions and commit are written to `paper/results/environment.txt` on each run. These steps were checked from a fresh clone in a new Python virtual environment, where every checksum matched.

```
git clone https://github.com/motionbenchmark/mb-core && cd mb-core
git checkout paper-v0.5
pip install maturin
(cd python && maturin build --release)
pip install "target/wheels/motionbench-*.whl[tracker]" scipy matplotlib
bash paper/reproduce.sh     # ~5 min
```

Expected outputs: the synthetic board is byte-identical to `results/leaderboard-synthetic-v0.1.json`; `paper/results/aim-sort-verdict.json` reproduces Table 6; `aim-sort-per-layout.json` reproduces Table 12; `aim-sort-failures.json` reproduces Table 7; `aim-sort-budget.json` reproduces Table 8; `index-sensitivity.json` reproduces Table 10; `capture-accuracy.json` reproduces Table 9 (small differences are possible across OpenCV/FFmpeg versions). SHA-256 checksums of the deterministic outputs are in `paper/results/SHA256SUMS` and are checked by the script.

Implementation details. All results were produced on a 2-core 2.1 GHz Intel Xeon virtual machine without a GPU; `reproduce.sh` takes about 4.5 minutes there. Simulated runs are deterministic given their layout and seed: run i uses seed $1000 + i$ on the public layout and $6000 + i$ on unseen layout `gen-`$(5000 + i \bmod 10)$; the class of twenty drivers uses seed 0, the bootstrap seed 0. The simulator steps at 10 ms and records poses every 50 ms.

B Secondary measures and the experimental index

Definitions. Path efficiency $e = \min(1, L_{\text{ref}}/L)$, with L the path driven and L_{ref} the reference length: for waypoints, the straight-line length from the start through the waypoints reached; for deliveries, the greedy tour that repeatedly goes to the nearest remaining object and carries it to its zone centre, truncated at the number delivered; for coverage, the area covered divided by the tool width. $e = 0$ when nothing was achieved. Consistency $c = \text{clamp}(1 - \sigma_S/\mu_S, 0, 1)$ with σ_S the sample standard deviation of success over runs, $c = 0$ when $\mu_S = 0$, and $\sigma_S = 0$ for a single run (which is why agents with fewer than five runs are unranked). Safety $s = 1/(1 + \bar{v}) \in (0, 1]$ with $\bar{v}$ the mean number of contact or rule violations per run. The index is

$$\text{MBI} = 0.50\, g + 0.20\, e + 0.15\, c + 0.15\, s \ \in [0, 1].$$

The weights are a design choice, not an estimate of what performance means.

When not to use it. The index should not be used across tasks, to compare agents whose goal scores differ clearly (the goal score already answers the question), or where consistency is high only because the score is uniformly low, since c does not distinguish the two.

Weight sensitivity. On the synthetic boards whose simulated agents respect their robots' limits, the index was recomputed for 5,000 weight vectors drawn near the defaults (Dirichlet with concentration $50\,w$) and uniformly from the simplex (Table 10). Under local changes the rankings are stable (median Kendall's $\tau \geq 0.94$); under arbitrary weights the coverage ranking is the least stable. Ranking by goal score alone agrees with the index at $\tau = 0.81$ to 0.91. This shows the ranking is insensitive to small changes in the weights, not that the weights are right.

		Local weights		Uniform weights	Goal only
Task	Agents	median τ	top agent same	median τ (5th pct.)	τ
MB-VEX-NAV@1.0	19	0.99	100%	0.95 (0.88)	0.91
MB-VEX-PICK@1.0	19	0.99	100%	0.90 (0.74)	0.81
MB-COVER@0.1	9	0.94	99.6%	0.83 (0.33)	0.83

Table 10: Kendall's τ between the default index ranking and rankings under perturbed weights (synthetic agents; `paper/analysis/index_sensitivity.py`).

C Simulated driver parameters

Table 11 lists the six characters. The class of twenty is drawn from a single skill level per student (Beta(2.2, 2.0)), from which each trait follows linearly with small noise (`class_roster` in `python/motionbench/game/driver.py`); the typical driver selected by the median of class means

has reaction 0.35 s, speed use 0.71, steering shake 4.6°, precision 0.66, decisions 0.81, mistakes 0.08 and hesitation 0.22. All values were set by hand to span a plausible range and have not been fitted to any person.

Character	Reaction (s)	Speed use	Shake (°)	Precision	Decisions	Mistakes	Hesitation	Strafe
Rookie Riley	0.55	0.55	9.0	0.30	0.55	0.16	0.25	0.0
Nervous Noah	0.45	0.60	5.0	0.60	0.70	0.07	0.40	0.1
Average Alex	0.40	0.70	5.0	0.55	0.75	0.08	0.15	0.2
Speedy Sloane	0.30	1.00	7.0	0.40	0.70	0.13	0.05	0.3
Steady Sam	0.35	0.55	3.0	0.85	0.85	0.03	0.10	0.3
Pro Priya	0.22	0.95	2.0	0.85	0.95	0.02	0.03	0.6

Table 11: Simulated driver characters. Decisions: probability of choosing the sensible next object; mistakes: probability per delivery of a fumble; hesitation: probability per decision of pausing.

D Unseen layouts, layout by layout

Table 12 groups the unseen-layout runs of Table 6 by layout (two runs per layout; the script `per_layout.py` in `paper/analysis` replays the same runs and adds none). The planner scores above the typical simulated driver on four layouts, the same on three and below on three; its per-layout means range from 1.5 to 4.0, against 1.0 to 3.0 for the vision routine.

Agent	1	2	3	4	5	6	7	8	9	10	All
Dead reckoning	0.5	0.0	1.5	1.5	0.0	1.0	0.0	2.0	0.0	0.0	0.65
Vision	3.0	3.0	3.0	3.0	2.5	3.0	1.5	1.0	2.0	1.5	2.35
Vision + tags	1.5	0.0	2.0	0.0	3.0	3.0	3.0	1.0	2.5	2.0	1.80
Planner	1.5	3.0	3.5	3.0	2.5	3.0	4.0	3.0	3.0	1.5	2.80
Typical sim. driver	3.0	2.0	3.0	2.0	3.0	3.0	3.5	3.0	3.0	3.0	2.85

Table 12: Mean points (of 6) per unseen layout, simulated.

E An example task and trace

A task specification (MB-VEX-NAV@1.0, one layout shown, waypoints shortened):

```
{ "id": "MB-VEX-NAV", "version": "1.0",
  "arena": { "width_m": 3.6576, "height_m": 3.6576,
             "obstacles": [ { "x": 1.6788, "y": 1.6788, "w": 0.30, "h": 0.30,
                              "label": "center-post" }, ... ], "zones": {} },
  "robot": { "radius_m": 0.229 }, "budget": { "time_s": 20 },
  "goal": { "kind": "waypoints", "pos_tol_m": 0.05, "heading_tol_rad": 0.0873 },
  "reference": { "floor": 0, "people_comparable": true },
  "layouts": { "dev-01": { "start": { "x": 0.45, "y": 0.45, "theta": 0 },
                "waypoints": [ { "x": 1.40, "y": 0.60, "theta": 0 }, ... ] } } }
```

A trace is a directory with `manifest.json` and `poses.csv` (and, when present, `events.csv` and `world_final.json`):

```
{ "spec_version": "0.1", "task": "MB-VEX-NAV@1.0", "seed": "dev-01",
  "agent": { "id": "team-auton-a", "class": "human_auton",
             "name": "Team auton A (odometry + GPS correction)",
             "meta": { "synthetic": true } },
  "trial": 0, "tier": "S", "capture": "synthetic",
  "files": { "poses.csv": "7d9322c4...bdde1" } }

t_s,x_m,y_m,theta_rad
0.000,0.454,0.451,-0.010
0.050,0.446,0.451,0.136
...
```

F Extension: multi-robot matches

The toolkit also includes a seeded 2-versus-2 match simulator, a renderer of broadcast-style video from a camera position measured on a real event broadcast, a per-robot report and a tracking benchmark. A baseline tracker finds 90–97% of robot positions with a median error of 7–9 cm on three simulated matches but switches identities 25–44 times per match (IDF1 [14] 48–66%). It is infrastructure for future work and is not used in the paper's conclusions.